

1. Investigación complementaria para defensa: componentes restantes del proyecto

1.1. Alcance

Este informe **no repite**:

- docs/Personal Research/data-structure-and-canonical-schema-research-report.md
- docs/Personal Research/qrdqn-research-report.md

Aquí se documentan los componentes que faltaban de cara al tribunal: componentes históricos, baseline clásico, tuning, scripts auxiliares y laboratorio.

1.2. 1) Mapa de “qué es cada cosa” (resto del proyecto)

Área	Archivo(s) clave	Qué es	Por qué importa en defensa
Branch histórico NSL-KDD Baseline no-RL	src/load_nsl_kdd.py, experiments/nslkdd_experiment.py src/baseline_random_forest.py	Línea histórica de benihemsk craspy por supervisado clásico (Random Forest)	Justifica evolución metodológica del TFG Permite defender que RL no se eligió “a ciegas”
Tuning de hiperparámetros	src/tune_hparams.py	Búsqueda con Optuna sobre QRDQN	Muestra metodología de optimización reproducible
Inferencia Fase 2 (legacy)	scripts/predict_real_traffic.py	Verificación antigua del pipeline	Sirve para explicar por qué existe v2 como ruta mantenida
Utilidades de robustez	src/scaling_utils.py	Clipping por percentiles y por z-score	Explica mitigación práctica de outliers/domain shift
Generación de tráfico de laboratorio	lab/docker/docker-compose.yml lab/docker/generator/generator.py	Modo aislado + generación de flujos	Demuestra estrategia de experimentación segura
Archivo de experimentos	experiments/README.md experiments/cicids2017-qrdqn-experiments/	Historial mantenimiento del sistema	Ayuda a separar evidencia actual de contexto histórico

1.3. 2) Algoritmos adicionales (aparte de QRDQN)

1.3.1. 2.1 Random Forest baseline

Dónde está: src/baseline_random_forest.py

- Modelo: RandomForestClassifier de scikit-learn.
- Configuración actual en código:
 - n_estimators=200
 - max_depth=None

- `n_jobs=-1`
- `random_state=42`
- `class_weight=None`

■ Flujo:

1. Carga dataset (CICIDS2017 o NSL-KDD) ya mapeado al contrato del proyecto.
2. Entrena RF.
3. Evalúa con matriz de confusión e informe de clasificación.
4. Guarda modelo `.joblib` en `models/`.

Mensaje para tribunal: este baseline es el “control clásico” para comparar con RL y demostrar que la línea RL aporta flexibilidad por función de coste (recompensas), no solo accuracy.

1.3.2. 2.2 Optuna (no es modelo, es algoritmo de búsqueda)

Dónde está: `src/tune_hparams.py`

- Crea un estudio con `optuna.create_study(direction="maximize")`.
- Objetivo: maximizar F1 de ataque.
- Espacio de búsqueda actual:
 - `learning_rate`: `1e-5` a `1e-3` (log)
 - `batch_size`: `{256, 512, 1024, 2048}`
 - `gradient_steps`: `{10, 50, 100}`
 - `net_arch`: `{[256,128], [512,256], [256,256]}`
 - `gamma`: `0.95` a `0.999`
 - `train_freq`: `{50,100,200}`
- Resultado se persiste en `runs/optuna/study_<timestamp>.json`.

Mensaje para tribunal: existe proceso formal de tuning; no es “prueba y error manual”.

1.4. 3) Parámetros clave que debes poder defender (resto de scripts)

1.4.1. 3.1 Validación leave-one-exact-CSV-out

Dónde está: `src/validate_leave_one_csv_out.py`

Parámetros CLI relevantes:

- `-timesteps` (default 30000)
- `-holdout-csvs` (elige folds concretos)

- `-max-rows-per-csv` (smoke/dev)
- `-predict-batch-size` (default 8192)

Qué añade frente a validación simple:

- Métricas por fold + agregados (`mean/std/min/max`).
- Métricas globales por suma de confusiones.
- Métricas operativas adicionales: `balanced_accuracy`, `specificity`, `fpr`, `fnr`, `block_rate`, `reward_per_sample`, tiempos de train/eval.

1.4.2. 3.2 Check B y Check C (detalles prácticos no obvios)

Dónde está: `src/validate_checks.py`

- **Check B (anti-leakage)** define umbral: `leakage_threshold = baseline_acc + 0.05`.
- **Check C (split duro por CSV/día)** reentrena QRDQN con hiperparámetros fijos de validación (no exactamente los mismos del train principal).
- Usa `ProgressCallback` para trazabilidad de entrenamiento en validaciones largas.

1.4.3. 3.3 Pipeline robusto de inferencia: utilidades de clipping

Dónde está: `src/scaling_utils.py`

- `apply_percentile_clipping(X, p_low, p_high)`: recorta features en bruto antes del scaler.
- `apply_z_clipping(X_scaled, max_z)`: recorta tras escalar a $[-max_z, +max_z]$.

Mensaje para tribunal: son defensas concretas contra colas extremas y drift de distribución en tráfico real.

1.5. 4) Funciones clave (resto no cubierto)

1.5.1. 4.1 `load_nsl_kdd_binary()`

Archivo: `src/load_nsl_kdd.py`

Qué hace:

- Descarga NSL-KDD con `kagglehub` si hace falta.
- Detecta columnas de etiqueta y dificultad.
- Binariza `normal` vs ataque.
- Permite modo canónico (con máscara) o modo legacy one-hot.
- Devuelve contrato estándar: `(X_train, y_train, X_test, y_test, scaler, feature_names)`

Por qué es defendible: es la pieza que conserva continuidad histórica del proyecto, pero separada del camino final de Fase 2.

1.5.2. 4.2 `compute_truth_metrics()` en inferencia v2

Archivo: `scripts/predict_real_traffic_v2.py`

Qué hace:

- Detecta columnas de verdad-terreno (`truth_y`, `truth_label`) cuando existen.
- Normaliza etiquetas a binario (0/1) y descarta filas no válidas.
- Calcula métricas de clasificación (`accuracy`, `precision/recall/f1`, `tp/tn/fp/fn`).
- Integra estas métricas en el `metrics.json` final de la ejecución.

Por qué importa: te permite defender evaluación controlada en Fase 2 cuando el CSV lleva etiquetas de referencia.

1.5.3. 4.3 `gen_traffic.py` (laboratorio)

Archivo: `lab/docker/generator/gen_traffic.py`

Qué hace:

- Genera dos patrones:
 1. múltiples conexiones cortas a puertos cerrados (“scan-like”)
 2. muchas peticiones HTTP con conexión nueva en cada request
- Añade `jitter` temporal para variabilidad.

Por qué importa: justifica cómo produces tráfico reproducible/controlado para pruebas de Fase 2.

1.6. 5) Red neuronal y arquitectura (solo lo complementario)

Sin repetir teoría QRDQN ya investigada:

- El pipeline principal usa `MlpPolicy` (MLP densa, no CNN/RNN).
- El repo también prueba arquitecturas alternativas en tuning (`[256,128]`, `[256,256]`, `[512,256]`).
- En validaciones y entrenamiento se mantiene acción discreta binaria (PERMIT/BLOCK), coherente con salida por acción del modelo.

Mensaje defendible: se eligió arquitectura MLP porque el input es vector tabular de flujo, no imagen/secuencia cruda.

1.7. 6) Dependencias técnicas y rol de cada una

Archivo: `requirements.txt`

- `numpy`, `pandas`: procesamiento numérico y tabular.
- `scikit-learn`: scaler, métricas, Random Forest.
- `gymnasium`: API del entorno RL.
- `torch`: backend de redes.
- `stable-baselines3` + `sb3-contrib`: algoritmos RL (incl. QRDQN).
- `tensorboard`: trazas de entrenamiento.
- `matplotlib`: soporte de visualización.
- `optuna`: tuning automatizado.

Mensaje para tribunal: stack estándar y auditable; no hay dependencias exóticas sin justificación.

1.8. 7) Legacy vs mantenido (muy útil en preguntas “trampa”)

- **Mantenido para Fase 2:** `scripts/predict_real_traffic_v2.py`
- **Legacy:** `scripts/predict_real_traffic.py` (carga datos de entrenamiento al import y no separa artefactos con el mismo nivel de robustez/CLI)

Cómo responder si preguntan por ambas: la v1 queda como referencia histórica/técnica; la v2 es la ruta oficial por diseño reproducible, CLI explícita y endurecimiento frente a domain shift.

1.9. 8) Guion corto para defensa oral (solo “resto”)

1. “Además del núcleo canónico + QRDQN, el proyecto tiene un bloque de **gobernanza experimental**: validación dura por folds exactos, agregación de métricas y archivado de runs.”
2. “Mantengo un baseline clásico (Random Forest) para comparación metodológica, no solo para maximizar métrica.”
3. “Tengo **tuning reproducible** con Optuna y búsqueda declarada de hiperparámetros.”
4. “La Fase 2 se apoya en un mini-lab aislado y scripts de tráfico controlado, no en pruebas opacas.”
5. “La inferencia v2 puede incluir métricas con verdad-terreno cuando existen etiquetas, mejorando la trazabilidad experimental.”

1.10. 9) Preguntas probables del tribunal y respuesta breve

¿Para qué conservar NSL-KDD si no es el baseline final? Para justificar evolución metodológica y comparar decisiones de diseño; el camino mantenido final es CIDS2017 + pipeline de Fase 2.

¿Por qué incluir Random Forest si el TFG es RL? Porque un baseline clásico ancla la comparación y evita conclusiones sin referencia.

¿Qué aporta leave-one-exact-CSV-out frente a métricas agregadas? Mide robustez por archivo real específico y reduce optimismo por mezcla de patrones entre train/test.

¿Qué pasa si en Fase 2 no hay `truth_label` o `truth_y`? La pipeline sigue funcionando en modo no supervisado operativo y reporta tasas de `block/allow`; las métricas supervisadas solo se añaden cuando hay verdad-terreno válida.

1.11. 10) Conclusión de esta investigación complementaria

Tus dos investigaciones cubren bien el núcleo (contrato de datos + QRDQN). Lo que faltaba para una defensa sólida era demostrar dominio del **ecosistema completo**: baseline clásico, tuning, validación avanzada por fold, tooling de laboratorio y distinción legacy/mantenido. Ese “resto” es exactamente lo que completa el relato técnico ante tribunal.